

AFRILANG Stdlib

std/c/parselog

Français. Bibliothèque AFRILANG 'std/c/parselog' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : logLen, logUpper, logLower, logTrim, logSplit, logJoin, logReplace.

'import "std/c/parselog"' · 'use parselog'

- 'logLen(s text) → number' - 'logUpper(s text) → text' - 'logLower(s text) → text' - 'logTrim(s text) → text' - 'logSplit(s text, delim text) → list text' - 'logJoin(parts list text, delim text) → text' - 'logReplace(s text, from text, to text) → text'

English. AFRILANG library 'std/c/parselog' (tier complex, category complex). Exports 7 function(s): logLen, logUpper, logLower, logTrim, logSplit, logJoin, logReplace.

'import "std/c/parselog"' · 'use parselog'

- 'logLen(s text) → number' - 'logUpper(s text) → text' - 'logLower(s text) → text' - 'logTrim(s text) → text' - 'logSplit(s text, delim text) → list text' - 'logJoin(parts list text, delim text) → text' - 'logReplace(s text, from text, to text) → text'

Catégorie / Category	Modules complex (c/)
Niveau / Tier	complex
Fonctions / Functions	7

June 16, 2026

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/parselog' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : logLen, logUpper, logLower, logTrim, logSplit, logJoin, logReplace.

```
'import "std/c/parselog"' · 'use parselog'
```

```
- `logLen(s text) → number`
- `logUpper(s text) → text`
- `logLower(s text) → text`
- `logTrim(s text) → text`
- `logSplit(s text, delim text) → list text`
- `logJoin(parts list text, delim text) → text`
- `logReplace(s text, from text, to text) → text`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/parselog"
use parselog
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- logLen(s text) → number•
logUpper(s text) → text•
logLower(s text) → text•
logTrim(s text) → text•
logSplit(s text, delim text) → list•
logJoin(parts list text, delim text) → text•
logReplace(s text, from text, to text) → text

4. Exemples d'utilisation

```
import "std/c/parselog"
use parselog
```

```
create result = logLen(/* args */)
say result
```

```
create result = logUpper(/* args */)
say result
```

```
create result = logLower(/* args */)
say result
```

```
create result = logTrim(/* args */)
say result
```

```
create result = logSplit(/* args */)
say result
```

```
create result = logJoin(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/parselog"`` en tête de fichier.
- Lancez ``afri-lang check`` avant de livrer.
- Combinez avec les modules `stdlib` de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module `parselog`

```
function logLen(s text) returns number
end
```

```
function logUpper(s text) returns text
end
```

```
function logLower(s text) returns text
end
```

```
function logTrim(s text) returns text
end
```

```
function logSplit(s text, delim text) returns list text
end
```

```
function logJoin(parts list text, delim text) returns text
end
```

```
function logReplace(s text, from text, to text) returns text
end
end
```

English

1. Overview

AFRILANG library 'std/c/parselog' (tier complex, category complex). Exports 7 function(s): logLen, logUpper, logLower, logTrim, logSplit, logJoin, logReplace.

```
'import "std/c/parselog"' · 'use parselog'
```

```
- `logLen(s text) → number`  
- `logUpper(s text) → text`  
- `logLower(s text) → text`  
- `logTrim(s text) → text`  
- `logSplit(s text, delim text) → list text`  
- `logJoin(parts list text, delim text) → text`  
- `logReplace(s text, from text, to text) → text`
```

2. Installation & import

Add the module to your program:

```
import "std/c/parselog"  
use parselog
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- logLen(s text) → number•
logUpper(s text) → text•
logLower(s text) → text•
logTrim(s text) → text•
logSplit(s text, delim text) → list•
logJoin(parts list text, delim text) → text•
logReplace(s text, from text, to text) → text

4. Usage examples

```
import "std/c/parselog"  
use parselog
```

```
create result = logLen(/* args */)   
say result
```

```
create result = logUpper(/* args */)   
say result
```

```
create result = logLower(/* args */)   
say result
```

```
create result = logTrim(/* args */)   
say result
```

```
create result = logSplit(/* args */)   
say result
```

```
create result = logJoin(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/parselog"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module `parselog`

```
function logLen(s text) returns number
end
```

```
function logUpper(s text) returns text
end
```

```
function logLower(s text) returns text
end
```

```
function logTrim(s text) returns text
end
```

```
function logSplit(s text, delim text) returns list text
end
```

```
function logJoin(parts list text, delim text) returns text
end
```

```
function logReplace(s text, from text, to text) returns text
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/parselog"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/parselog/>

Source (extrait)

```
module parselog

function logLen(s text) returns number
end

function logUpper(s text) returns text
```

```
end

function logLower(s text) returns text
end

function logTrim(s text) returns text
end

function logSplit(s text, delim text) returns list text
end

function logJoin(parts list text, delim text) returns text
end

function logReplace(s text, from text, to text) returns text
end
end
```